

软件开发技术

第3章 软件开发过程

浙江大学 机械工程学院 航空制造工程研究所

王青 李江雄

- 1 软件生命周期
- 2 软件开发过程模型
- 3 统一软件开发过程



01

软件的生命周期

软件生命周期由软件定义、软件开发、运行维护三个时期组成。



(a) 软件定义时期

任务：

- ◆确定软件开发工程必须完成的总目标
- ◆确定工程的可行性
- ◆导出实现工程目标应该采用的策略及系统必须完成的功能
- ◆估计完成该工程需要的资源和成本，制定工程进度表

子阶段：

- ◆问题定义
- ◆可行性研究
- ◆需求分析

责任者

- ◆系统分析员

(b) 软件开发时期

任务

- ◆ 具体**设计**前一时期定义的软件
- ◆ 根据设计**实现**前一时期定义的软件

子阶段

- ◆ 总体设计
- ◆ 详细设计
- ◆ 编码和单元测试
- ◆ 综合测试

总体设计与详细设计合称为系统设计

编码和测试阶段称为系统实现

(c) 运行维护时期

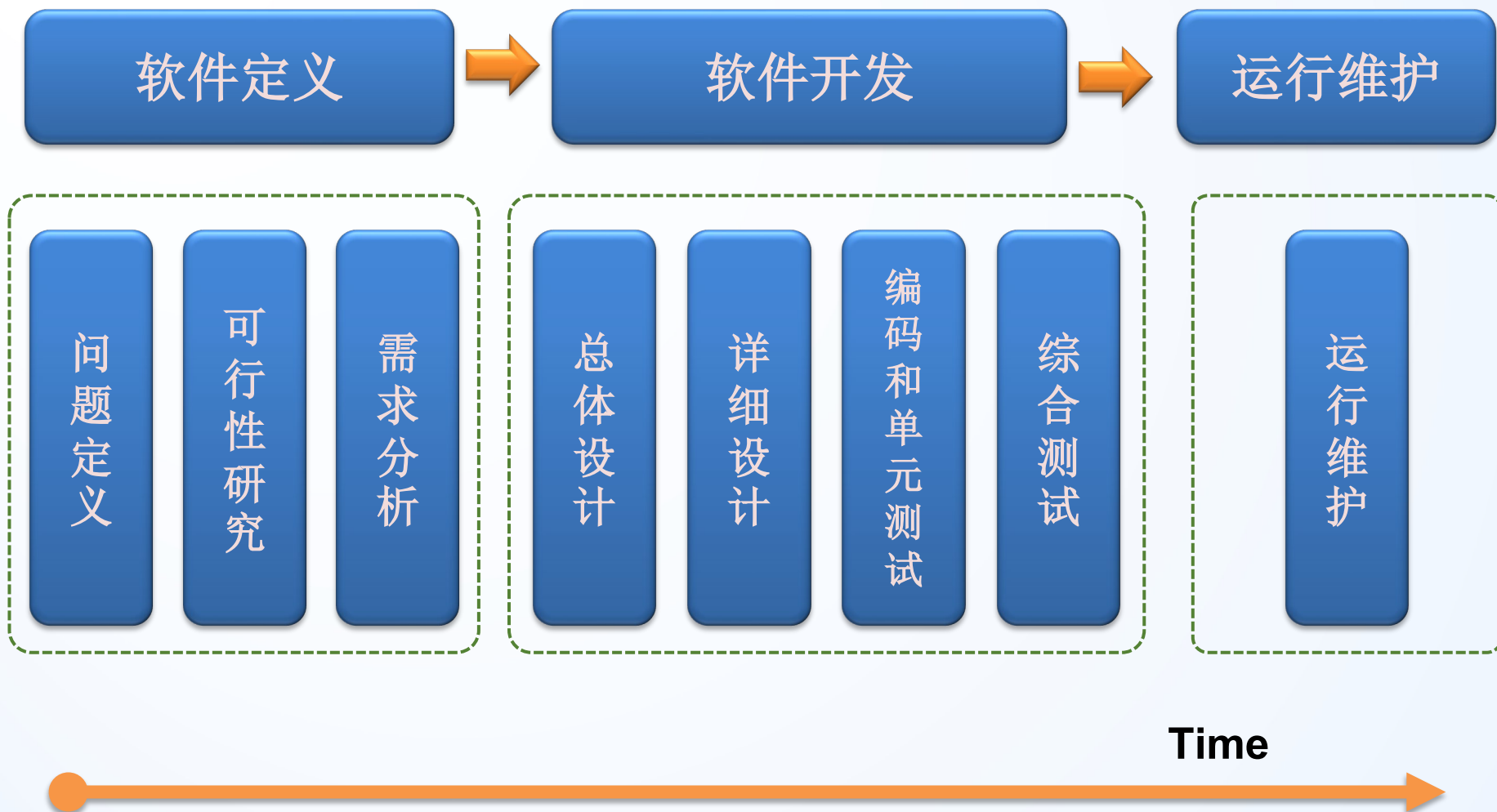
任务

- ◆ 使软件持久地满足用户的需要

子阶段

- ◆ 每一次维护活动本质上都是一次压缩和简化的定义和开发过程

- 当软件在使用过程发现错误时，应该加以改进
- 当环境改变时，应该修改软件以适应新的环境
- 当用户有新的要求时，应该及时改进软件以满足用户的新需要



◆ 问题定义

问题定义阶段必须回答的关键问题是：“**要解决的问题是什么？**”

如果不知道问题是什么就试图解决这个问题，显然是盲目的，只会白白浪费时间和金钱，最终得出的结果很可能毫无意义的。尽管确切地定义问题的必要性是十分明显的，但是在实践中它可能最容易被忽视的一个步骤。

通过对客户的访问调查，系统分析员扼要地写出关于问题性质、工程目标和工程规模的书面报告。在经过讨论和必要的修改之后，报告应该得到客户的确认。

系统目标和规模说明书 1996年3月

1.项目：教材销售系统

2.问题：人工发售教材手续繁琐且易出错。

3.项目目标：建立一个高效率、无差错的微机教材销售系统。

4. 项目规模：利用现有微机，软件开发费用不超过3000元

5.初步想法：建议在系统中增加对缺书的统计与采购功能。

6.可行性研究：建议进行六天的可行性研究，研究费用不操作200元。

◆ 可行性研究

该阶段要回答的问题是：“对于所定义的问题有行得通的解决方法吗？”

包括：是否值得去解决，是否有方法解决。
此阶段，系统分析员需要在较抽象的高层次上进行分析 and 设计，回答上述问题。

可行性研究的结果是客户做出是否继续进行该工程的决定的重要依据。只有投资可能取得较大效益的工程才值得继续进行下去。

对于可行的软件项目要进行开发，必须要审定项目的开发计划，估算费用，确定资源分配和项目开发的速度安排，这就需要制定出软件项目的开发计划。

可行性研究报告

项目开发计划

以上两环节又称为“计划阶段”

◆ 需求分析

准确的确定“**为了解决该问题，目标系统必须做什么**”，主要是确定目标系统必须具备哪些功能。

系统分析员在需求分析阶段必须解决以下矛盾：

- 用户了解他们面对的问题，知道必须做什么，但是通常不能完整、准确地表达他们的要求，更不知道如何利用计算机解决他们的问题；
- 软件开发人员知道如何用软件实现人们的要求，当对用户的特定任务的具体要求并不清楚。

系统分析员必须和用户密切配合，充分交流信息，以得到经过用户确认的系统逻辑模型，通常用数据流图、数据字典和简要的算法表达系统的逻辑模型。

在需求分析阶段确定的系统逻辑模型是以后设计和实现目标系统的基础。

因此必须准确完整地体现用户的要求。这个阶段的一项重要任务就是建立需求规格说明书，准确地记录对目标系统的需求，这份文档通常称为规格说明书。

至此软件定义
完成

◆ 总体设计

这个阶段必须回答的关键问题是：“**概括地说，应该怎样实现目标系统？**”，因而总体设计又称概要设计。

- (1) 要设计出实现目标系统的可能方案；
- (2) 分析每种方案的优缺点，推荐一个最佳方案
- (3) 制定实现最佳方案的详细计划
- (4) 如果客户接受最佳方案，则为该方案设计程序的体系结构，即确定程序由哪些模块组成、各模块间关系如何。

概要设计说明书

◆详细设计

回答的关键问题：“**应该如何实现这个系统呢？**”

详细设计阶段的任务是要把解决问题的方法具体化，但不是编程序，而是“绘出程序的蓝图”，即设计出程序的详细规格说明，该规格说明详细描述了程序的必要细节。程序员根据该规格说明可以写出程序代码。

详细设计又称模块设计，它注重于对每个模块的设计。每个模块的设计包括模块的算法和数据结构。

总体设计与详细设计，合称为“设计阶段”。

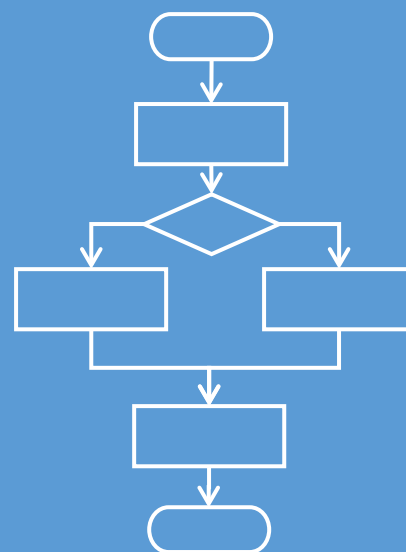
详细设计说明书

模块n

输入：

输出：

逻辑：



◆ 编码和单元测试

这个阶段的关键任务是：写出正确的、容易理解的、容易维护的程序模块。

程序员应该根据目标系统的性质和实际环境，选取一种适当的高级程序设计语言（当然也可以用汇编语言），

把详细设计的结果变为用计算机语言写就的程序，

然后仔细测试编写的每一个模块。

程序清单

```
void main (
{
double a;
double b;
double c;
c=a+b;
.....
}
```

◆ 综合测试

关键任务：通过各种类型的测试（及相应的调试）使软件达到预定的要求。

最基本的测试是集成测试和验收测试：

1. **集成测试**：根据软件结构，把经过单元测试检验的模块按某种选定的策略装配起来，在装配过程中对程序进行必要的测试。
2. **验收测试**：按照规格说明书的规定，由用户对目标系统进行测试及验收。

通过软件测试结果的分析可以预测软件的可靠性；反之对软件可靠性的要求可以决定测试和调试过程什么时候可以结束。

应该用正式的文档资料把测试计划、详细测试方案以及实际测试结果保存下来，作为软件配置的一个组成部分。



测试报告

◆运行维护时期

一般情况下有四类维护活动：

- (1) **改正性维护**：诊断和改正在使用过程中发现的软件错误；
- (2) **适应性维护**：修改软件以适应环境变化；
- (3) **完善性维护**：根据用户的要求改进或扩充软件使它更完善；
- (4) **预防性维护**：修改软件为将来的维护活动预先做准备。

提出维护要求

分析维护要求

提出维护方案

审核维护方案

确定维护计划

修改软件设计

修改程序

测试程序

复查验收

文档记录



02

软件开发 过程模型

软件开发过程是指为了获得高质量软件所需要完成的一系列任务的框架，它规定了完成各项任务的工作步骤。

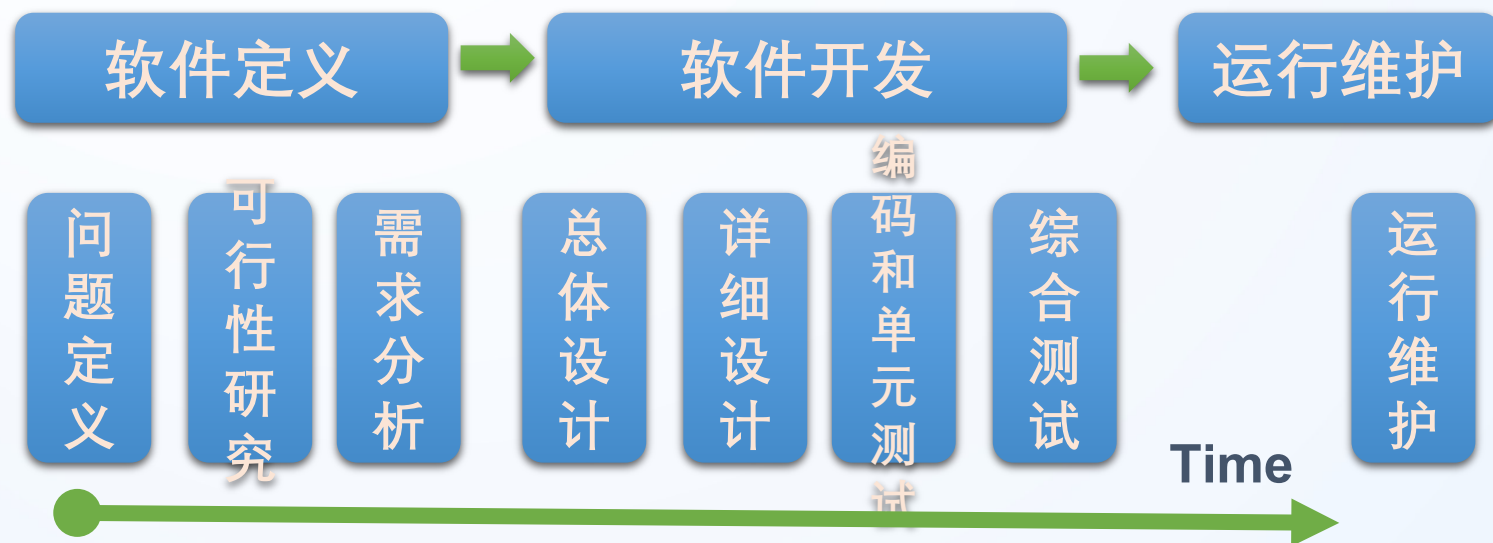
软件开发过程定义了各个步骤中应该交付的文档资料，为保证软件质量和协调变化所需要采取的管理措施，以及标志软件开发各个步骤任务完成的里程碑。

一般用软件生命周期模型来描述软件过程。

软件生命周期模型规定了生命周期的阶段划分以及各个阶段的执行顺序。

◆理想软件过程

- 首先明确客户的需求（即做问题定义和可行性分析）
- 再进行分析（需求分析）
- 在得到完整的分析结果之后，进入设计阶段
- 紧接着进行软件产品的编码实现和单元测试
- 然后，进行综合测试，
- 最后，将软件安装到客户计算机上，运行软件

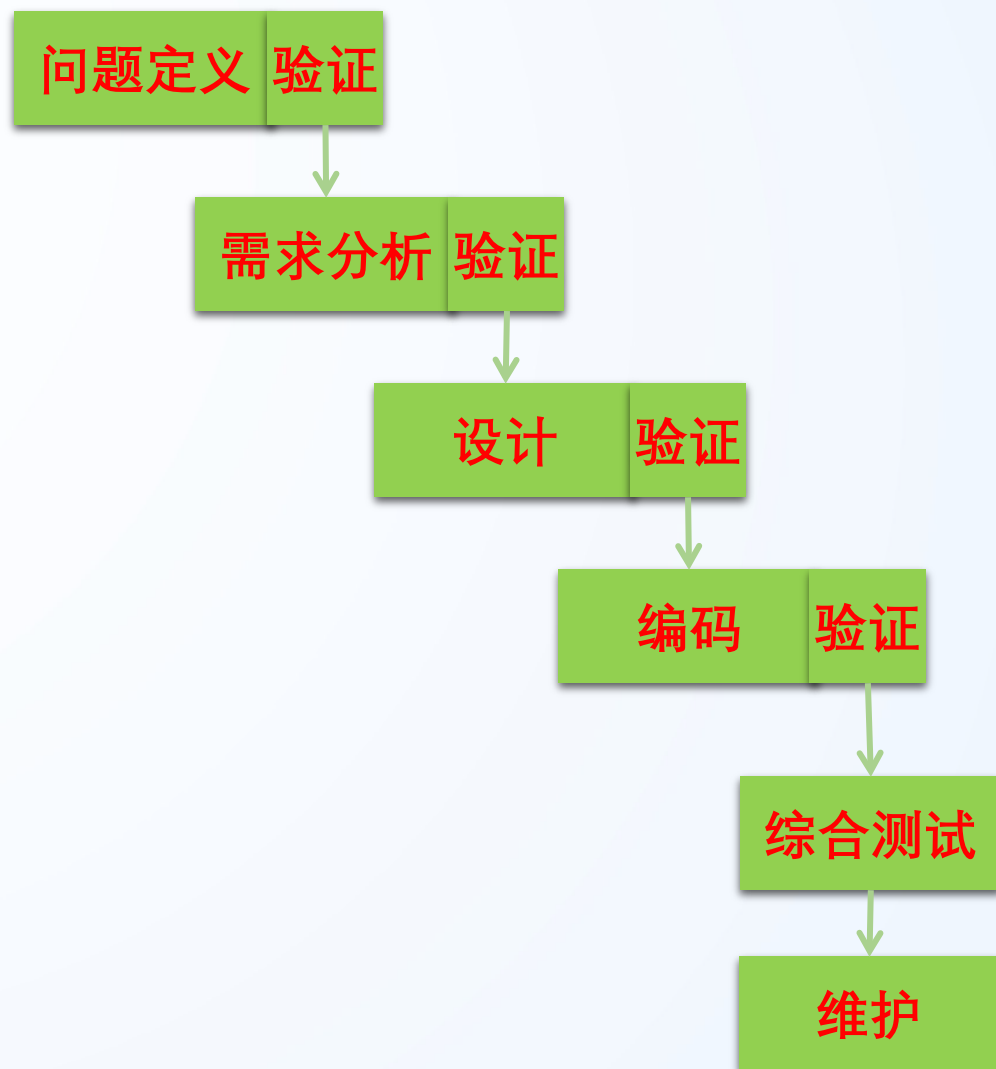


◆瀑布模型

瀑布模型是软件工程中广泛使用的生命周期模型，传统软件工程的开发过程，基本上都是使用瀑布模型来描述。

特点：

1. 阶段间具有顺序性和依赖性
2. 推迟实现的观点
3. 质量保证的观点



特点1：阶段间具有顺序性和依赖型

(a) 必须等前一阶段的工作完成之后，才能开始后一阶段的工作

(b) 前一阶段的输出文档就是后一阶段的输入文档，因此，只有前一阶段的输出文档正确，后一阶段的工作才能获得正确的结果。

特点2 推迟实现的观点

瀑布模型在编码之前设置了系统分析与系统设计的各个阶段，这些阶段的基本任务中不包含软件的物理实现（即编码），只是要求充分明确问题、建立正确的目标系统逻辑模型。尽可能在编码前将所有的问题都弄清楚了才开始编码。使得编码工作的基础扎实，降低的编码风险。

清楚地区分逻辑设计与物理设计，尽可能推迟程序的物理实现，是按照瀑布模型开发软件的一条重要指导思想。

特点3 质量保证的观点

软件工程的基本目标是优质、高效。为了保证所开发的软件的质量，在瀑布模型的每个阶段都应坚持几个重要做法：

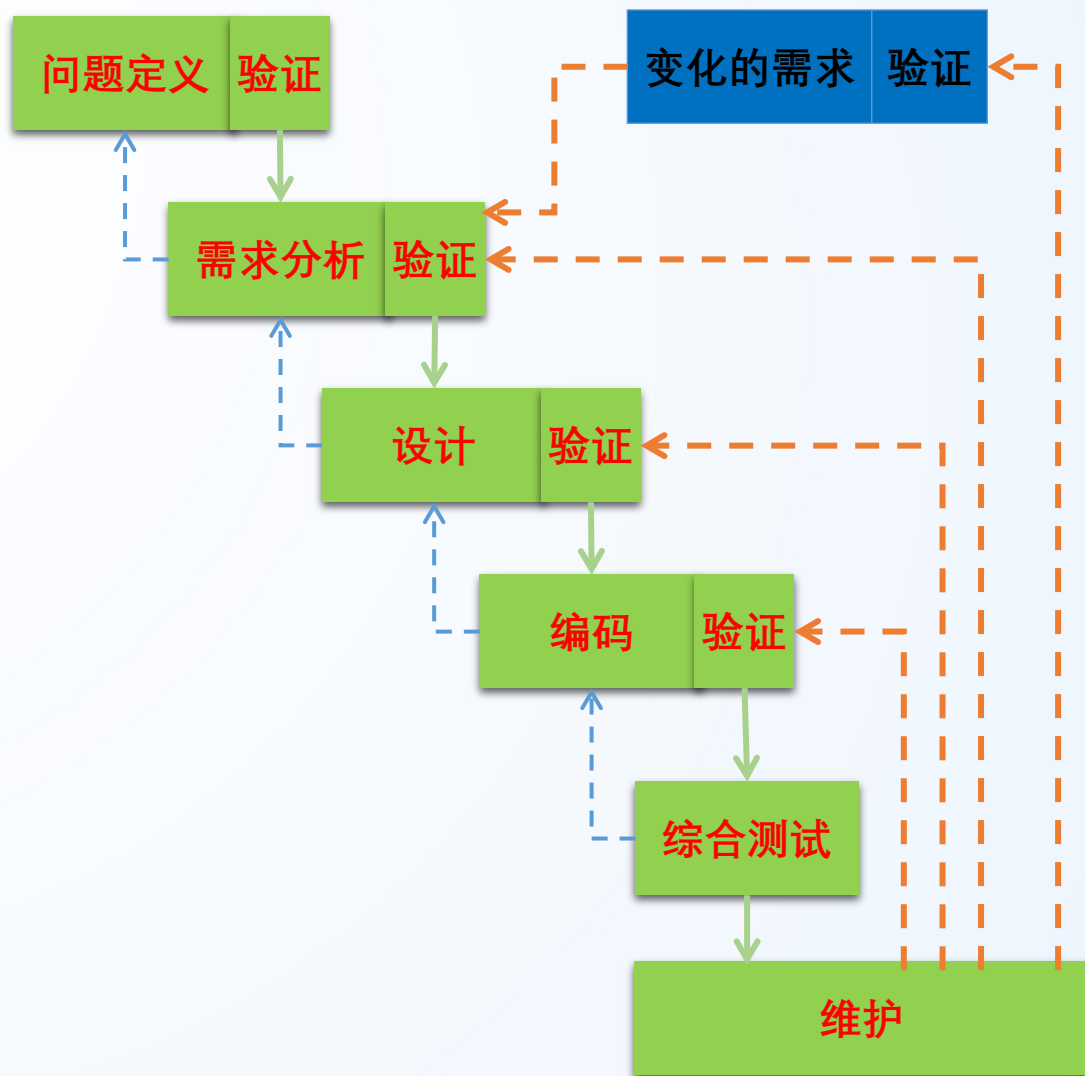
- ◆ 每个阶段都必须完成规定的文档，没有交出合格的文档就是没有完成该阶段的任务。
- ◆ 完整、准确的合格文档不仅是软件开发时期各类人员之间相互通讯的媒介，也是运行时期对软件进行维护的重要依据。
- ◆ 每个阶段结束前都要对所完成的文档进行评审，以便尽可能早发现问题，改正错误。
- ◆ 越是早期阶段犯下的错误，曝露出来的时间越晚，排除故障改正错误所需付出的代价也就越高。因此，及时审查，是保证质量，降低软件成本的重要措施。

传统的瀑布模型实际上也是理想软件开发过程模型，没有反映软件开发的实际情况。

◆ 改进的瀑布模型

考虑到在软件开发的后续阶段中，可能发现前面阶段的错误，需要重返前面阶段做修改的特点，在传统瀑布模型中加上“反馈环节”形成改进的瀑布模型。改进的瀑布模型与实际软件开发过程更相符，它就是今天所说的瀑布模型。

如图白线箭头所示，当在后面阶段发现前面阶段的错误时，需要沿反馈线返回到前面阶段，修正前面阶段的制品（如文档）后回来继续完成后面阶段的任务。



依照瀑布模型组织软件开发过程，有如下优点：

可强迫开发人员采用规范的方法(如结构化方法、面向对象方法)；

严格地规定了每个阶段必须提交的文档；

要求每个阶段交出的所有制品都必须经过质量保证小组的仔细验证。

瀑布模型的成功在很大程度上是由于它基本上是一种文档驱动的模式。各个阶段产生的文档是维护软件产品时必不可少的，没有文档的软件几乎是不可维护的。遵守瀑布模型的文档约束将使得软件维护变得比较容易，因而能显著降低软件开发费用。经统计：软件开发的2/3以上的费用花在维护阶段。

缺点：

- 在软件产品交付给用户之前，用户只能通过文档来猜测产品是什么样子。有时很难让用户真正了解产品，从而用户对产品是否符合其要求的判断容易出错。

◆快速原型模型

在问题定义阶段，尽快建立软件原型，让用户能尽早判断问题定义的准确性。

- 利用该原型与用户讨论目标软件的功能是否符合其要求。
- 用户可通过试用，了解目标软件的概貌、评价开发人员对问题理解的正确性，提出修改意见。
- 开发人员根据修改意见修改原型，直到用户确认该原型系统确实能如实反映他们的需求。
- 开发人员便可据该快速原型进行进一步的需求分析、设计等后续工作。

快速原型作为一种工作模型，在功能上反映用户的主要需求，是目标软件一个功能子集。快速原型的用途是获知用户的真正需求，一旦需求确定了，原型将被抛弃。

因此，开发人员应该尽可能快地构造出原型系统，不应纠结原型的内部结构，能准确反映用户关键需求即可。当然，原型系统还应能适应快速修改。

快速原型模型主要优点在于：

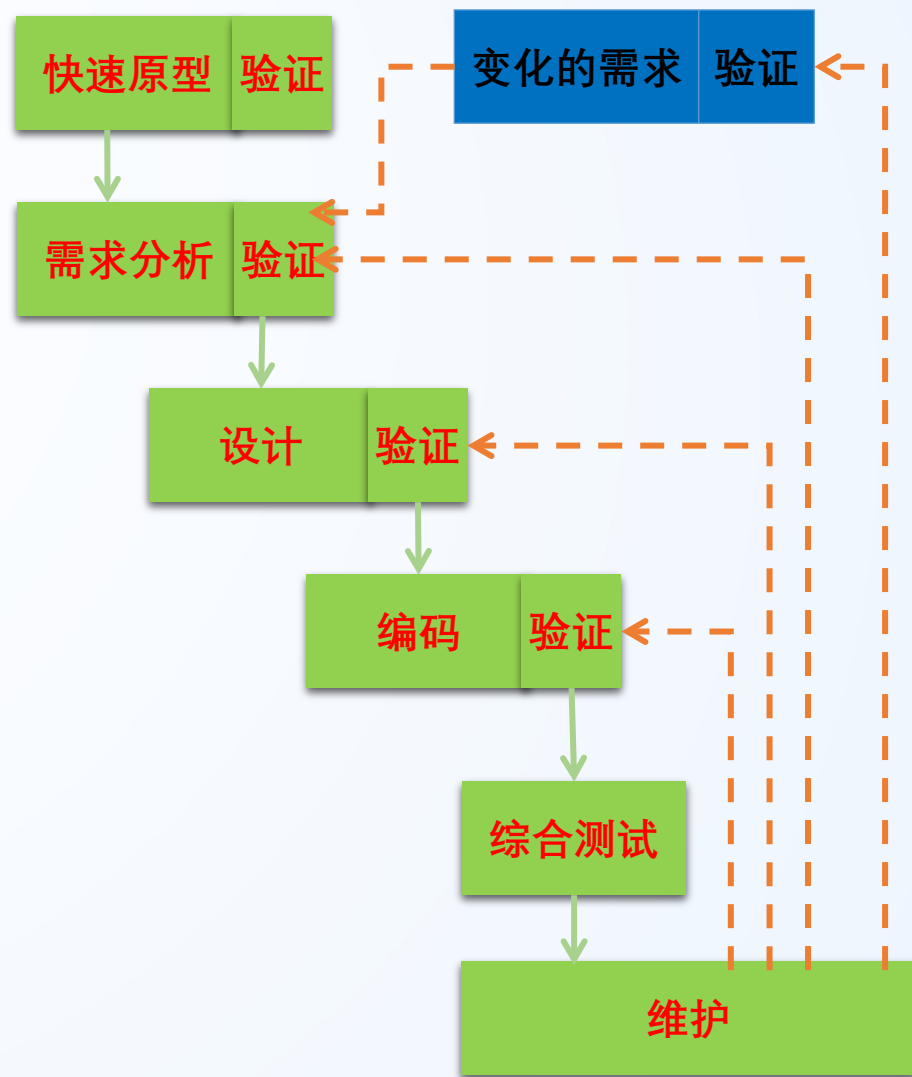
产品开发过程中从快速原型到可交付产品的处理过程是线性的；在快速原型中不太可能需要瀑布模型中的反馈循环。

原因：

(1) 原型系统已经得到了用户确认，据此得到的软件规格说明文档可以被期望为是正确的。

(2) 通过开发原型系统，开发人员已经学到了较多的东西，在设计阶段犯错的可能性大大降低。

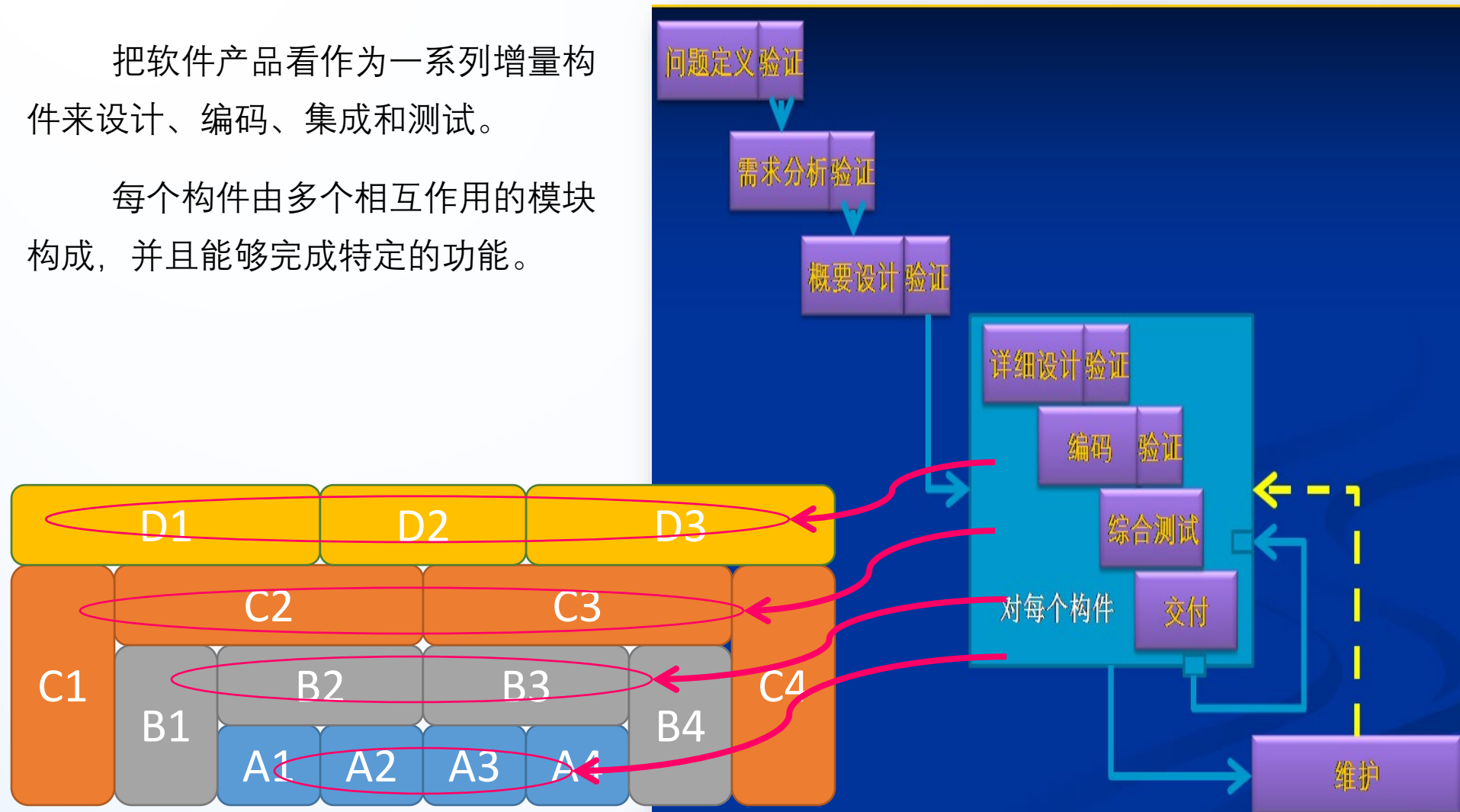
(3) 在开发原型系统时，已经积累了一定的经验，使得编码实现时犯错的可能性也大大降低。



◆ 增量模型（渐增模型）

把软件产品看作为一系列增量构件来设计、编码、集成和测试。

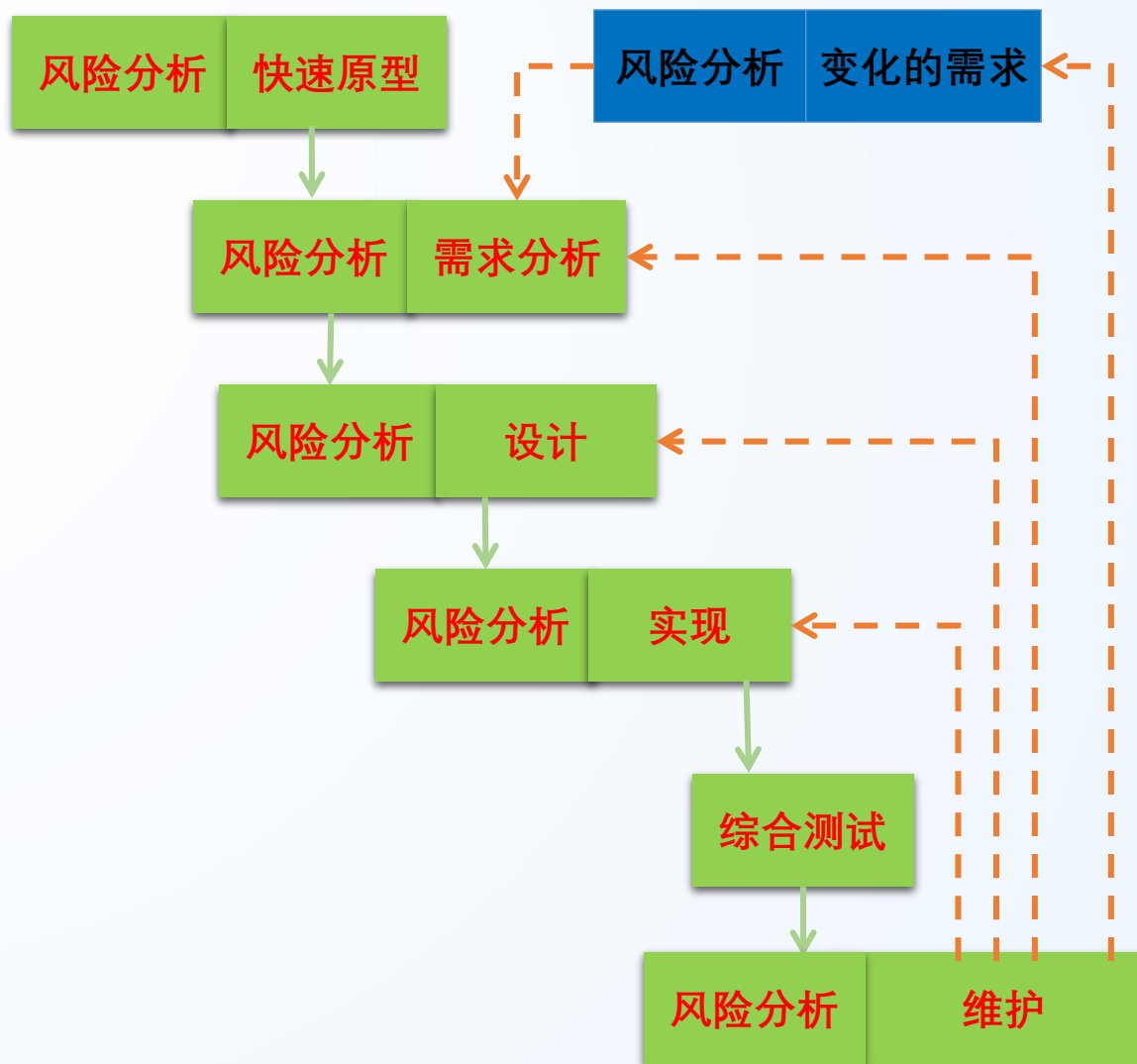
每个构件由多个相互作用的模块构成，并且能够完成特定的功能。



◆ 螺旋模型

螺旋模型的宗旨是尽可能降低软件开发风险。

其方法是以快速原型模型为基础，在软件开发生命周期各阶段之前，都增加风险分析。



完整的螺旋模型如右图所示。

每个开发阶段对应图中一圈（四个象限）

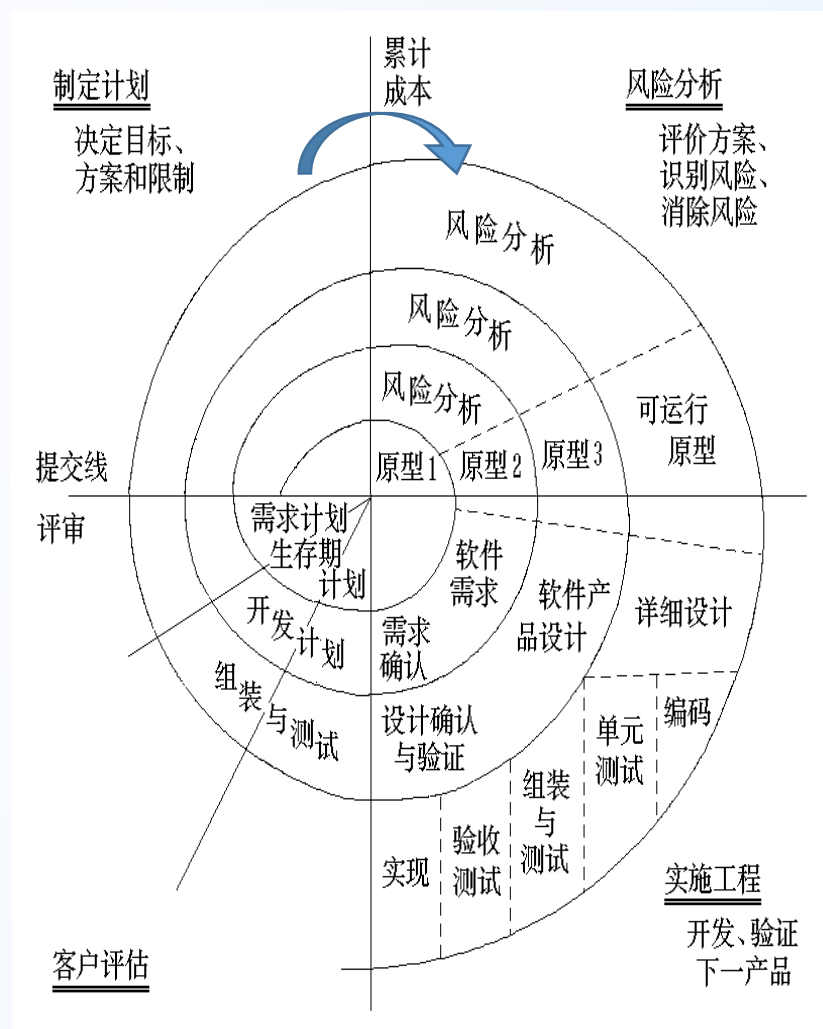
每个开发阶段都需要：

制定计划（确定阶段目标、方案、约束）

风险分析（通过建造概念模型来评价方案、识别风险、消除风险）

阶段工作实施（阶段软件开发，按瀑布模型的方式）

评估本阶段、计划下阶段





03

统一软件开发过程

3.1 统一软件开发过程

软件过程是软件的开发方式，是为了获得高质量的软件产品需要完成的一系列任务的框架。它规定了完成各项任务的工作步骤。前面所介绍的软件生命周期模型，实际是就起到了描述描述不同软件过程的作用，每种生命周期模型代表者一种软件过程。所以也称软件过程模型。

统一过程是Rational软件公司（三剑客：**Grady Booch**、**Ivar Jacobson**、**Jim Rumbaugh**）推出的一种完整而且完美的软件过程，称Rational Unified Process, RUP;

主要适合面向对象软件开发范型;

RUP总结了经过多年商业化验证的6条最有效的软件开发经验，这些经验被称为“最佳实践”。

它们是统一软件过程的依据，或者目的。

同时，这6个“最佳实践”也是统一开发过程的特点。



G. Booch



J. Rumbaugh



I. Jacobson

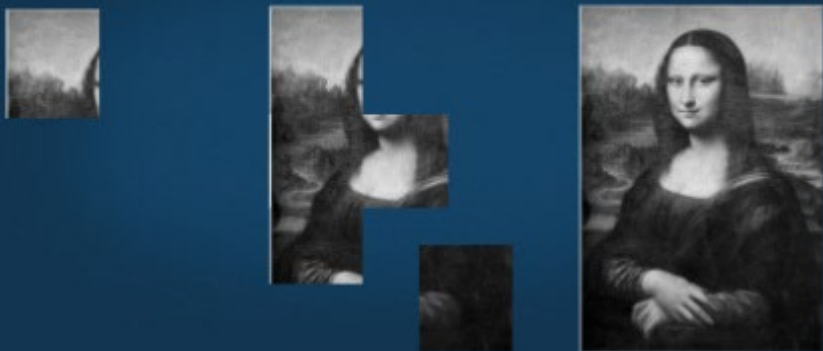
3.2 统一软件开发过程基本原理（最佳实践）

（1）迭代式开发

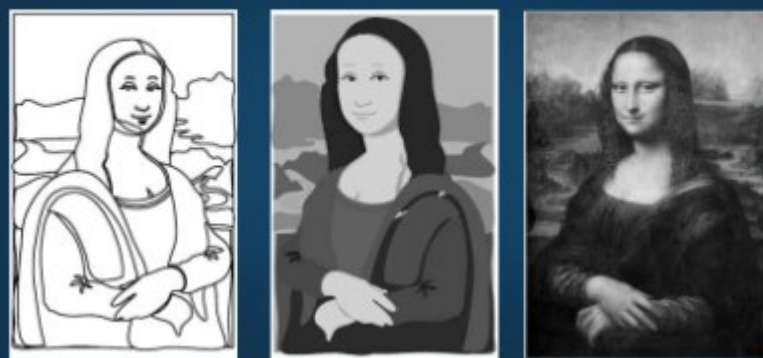
由于在整个软件开发过程中，客户的需求会经常改变，因此需要有一种能够通过一系列细化、若干个渐进的反复过程而得出有效解决方案的迭代方法。在每次迭代过程中，需求都可能有变化，开发者可以在迭代过程中，一方面细化原有的分析、设计制品，一方面根据需求变化修改原有的需求分析制品、原有设计制品。

面向对象软件开发方法（面向对象范型）总是使用建模方法，在统一过程中，使用UML来表达软件各个阶段的模型，迭代式开发过程就是一个不断修改和丰富软件的UML模型的过程。

增量开发



迭代式开发



(2) 管理需求

在软件开发过程中，客户需求将不断发生变化，因此确定系统的需求是一个连续的过程。RUP描述了如何提取、组织系统的功能需求和约束条件并把它们文档化。

RUP使用用例 (USE CASE)和脚本来捕获软件的功能性需求，并由用例来驱动设计和实现。

(3) 使用基于构件的体系结构

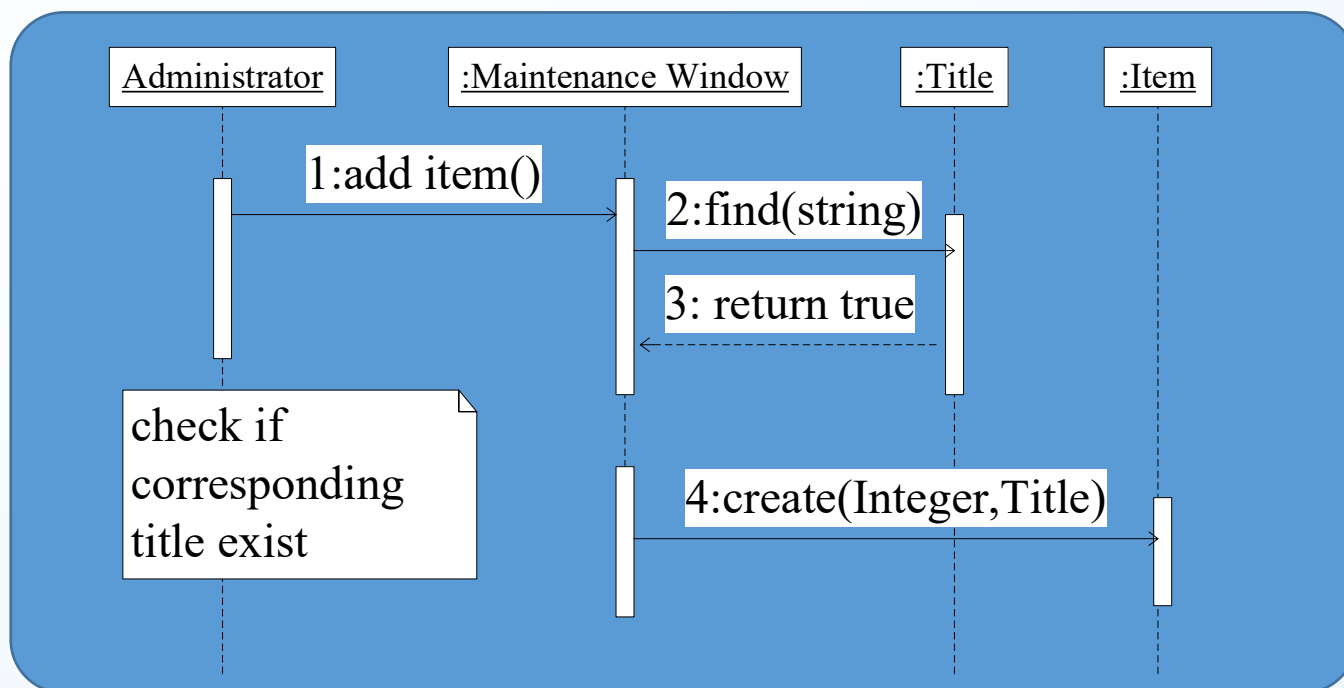
所谓构件就是功能清晰的模块或子系统。系统可以由已经存在的、由第三方法开发商提供的构件组成，因此构件使软件重用成为可能。

RUP提供了使用现有的或新开发的构件定义体系结构的系统化方法，从而有助于降低软件开发的复杂性，提高软件重用率。

(4) 可视化建模

为了更好地理解问题，人们常常采用建立问题模型的方法。所谓模型就是为了理解事物而对事物做出的一种抽象，是对事物的一种无歧义的书面描述。由于应用领域不同，模型可以有文字、图形或数学表达式等多种形式。一般认为，可视化的图形形式更有助于理解。

RUP采用UML作为建模语言，在软件开发过程中对软件的各个方面进行建模。UML是一种可视化的建模工具，它由一系列图形符号组成。



(5) 验证软件质量

实践表明，在软件投入使用后再去查找和修改出现的问题，比在开发的早期阶段就进行这项工作需要花费更多的人力和时间。

在RUP中，软件质量评估不再是事后型的或由单独小组进行的孤立的活动的，而是内建在贯穿于软件开发过程、由全体软件开发成员参与的所有活动中。

(6) 控制软件变更

软件开发过程中，软件开发制品变更是不可避免的。为了对软件开发进行有效管理，管理软件的变更必不可少。只有有效管理了软件变更，才能确保每个修改都可跟踪、可复查。

RUP描述了如何控制、跟踪和监控软件修改，以确保迭代开发成功。

3.3 RUP的核心 workflow

RUP有9个工作流，其中业务建模、需求、分析与设计、实现、测试、部署为核心工作流，配置与变更管理、项目管理、环境为核心支持工作流。

(1) 业务建模 (Business Modeling) 工作流

深入了解使用目标系统的机构及商业运作，评估目标系统对使用它的机构的影响。

包括：评估业务状态、确定业务流程、改进业务流程、设计业务流程实现、改进角色和职责、流程自动化研究等工作。

(2) 需求 (Requirements) workflow

需求 workflow 的目标是使开发组织明确用户的需求。（用户到底要什么？要让用户知道，同时要让开发人员知道）

需求 workflow 的输出必须为客户所完全理解，换句话说就是必须把需求 workflow 的制品表达成客户语言，即人类使用的自然语言。

包括：

- 开发团队需要从根本上理解软件的应用领域（简称：领域），即目标软件产品将要运行的特定环境。如工业企业管理层、车间、银行、商店等等。
- 要与用户在对软件产品的功能需求的理解上充分一致；
- 要证明软件成本是合理的；
- 要了解系统的约束条件，如最后期限、可靠性、成本，有时还有对可执行的目标程序的大小有限制（如运行在飞机机载计算机上的软件、运行在单片机上的软件）、运行速度约束等等。

具体而言，需求工作流的目的在于：

- 与客户或其他涉众在系统的工作内容方面达成并保持一致；
- 使系统开发人员能够更清楚地了解系统需求；
- 定义系统边界（限定）；
- 为计划迭代的技术内容提供基础；
- 为估算系统开发成本和时间提供基础；
- 定义系统的用户界面，重点表达是用户的需要和目标。

(3)分析 workflow

分析 workflow 的目标是分析并精化需求，以易于软件正确开发和易于维护所必需的需求细节。

分析 workflow 的目的在于：

- 采用精确的、无二义的语言来描述用户需求，以确保设计和实现 workflow 的正确实施。
- 在实施分析 workflow 期间，添加更多的细节，这些细节可能与用户理解目标软件产品无关，但是对于软件开发人员而言是必要的。例如状态图中的初始状态肯定不为客户所关注，但开发软件却必不可少。

分析 workflow 输出是软件的规格说明文档（在 RUP 中是一系列的 UML 制品），较之于自然语言，这些 UML 制品形成了对软件的更为精确的描述。

在面向对象范型中，分析 workflow 通过领域分析，将软件产品分解为相互独立的组件（类），提取出组件的属性、方法及组件之间的关系，然后在软件规格说明文档中明确下来。方法的实现在设计 workflow 中确定。

为什么将获取客户需求分为两个独立的工作流？

与客户进行需求获取需使用客户可以理解的自然语言和领域内术语，但自然语言无一例外都或多或少不够严密，容易引起误解，而如果用数学符号来陈述需求，就不会混淆但妨碍客户理解需求，造成客户与开发者之间对需求形成误解，最终导致开发出的软件产品不是客户所需求。

解决办法：

采用两个独立的工作流，需求工作流按客户的语言来表达，分析工作流则采用更为精确的语言，以保证设计和实现工作流的正确实施。

(4) 设计 workflow

产品的规格说明详细给出了产品的功能，设计 workflow 则说明了如何实现产品的这些功能。更准确地说，设计 workflow 的目标是精化分析 workflow 制品，直到程序员能够以某种形式实现。

在设计 workflow 中，开发人员确定产品的内在结构，提取出方法，并分配给相应的组件（类）。特别地，必须详细制定每个方法的接口（即传递给方法的参数以及从方法返回的参数）和合适的实现算法。

组件属性的提取是在分析 workflow 中完成，但是，组件属性的格式则在设计 workflow 中确定。例如在分析 workflow 中提取了属性 grossProfit(总利润)，在设计 workflow 中确定该属性表示为8位整数。

(5) 实现 workflow

用选定的程序语言，把设计模型转换为实现结果。包括：

- ◆ 形式化定义代码结构；
- ◆ 用构件实现类和对象；
- ◆ 对开发出的构件进行单元测试；
- ◆ 把不同实现人员开发的模块集成为可执行的系统等等。

(6) 测试 workflow

在统一软件过程中，自始至终，测试与其他 workflow 并行实施。

(a) 需求制品测试

在软件产品的整个生命周期中，若要能对需求制品进行测试，则软件的整个生命周期的所有制品都必须具有“可追溯性”，如根据某个实现制品找到相对应的设计制品、根据某个设计制品找到相对应的分析制品、根据分析制品找到相对应的需求制品。

(b) 分析制品测试

分析制品主要是软件的规格说明文档，分析人员和质量保证小组应尽可能早地找出规格说明中的错误。校验规格说明错误的最好方法是评审会议。由分析人员和用户方出席。由评审员从头到尾审查分析制品，以验证是否存在错误。

走查和检查是评审的两种形式。

(c) 设计制品测试

检查规格说明的每部分是否都与设计的某些部分相对应；

同样，一般采用评审会议的方法对设计制品进行评审，此时，用户可以排除在外。

在整个设计过程中，对于每个独立的设计制品，开发人员与质量保证小组一起工作，以确保设计的正确性。

主要检查内容包括：

- 逻辑错误
- 接口错误
- 缺少异常处理（错误情况的处理）
- 以及与规格说明不一致（最为重要）

(d) 实现测试

- 检查各个子系统的交互与集成
- 验证所有需求是否都被正确地实现
- 识别、确认缺陷并确保在软件部署之前消除缺陷

(7) 维护 workflow

交付后维护并非产品被交付并安装到客户计算机之后才勉强实施的行为，从软件项目一开始就要进行计划。只要是可行的设计，就必须考虑未来的改进。

交付后维护的一个常见问题在于文档化，或者是缺少文档。

文档：

- 帮助维护人员理解产品；
- 可用于对改进、改正的内容进行追溯，以达到全面修改的目的；
- 使用原先测试用例进行全面测试，以确保其他内容没有受到影响；（回归测试）
- 记录所做的修改，及修改理由。

(8) 部署 workflow

生成目标系统的可运行版本并将软件移交给最终用户。

(9) 配置和变更管理

跟踪并维护在软件开发过程中产生的所有制品的完整性和一致性；

(10) 项目管理

提供项目管理框架，为软件开发项目制定计划、人员配备、执行和监控等方面的实用准则，并为风险管理提供框架；

(11) 环境

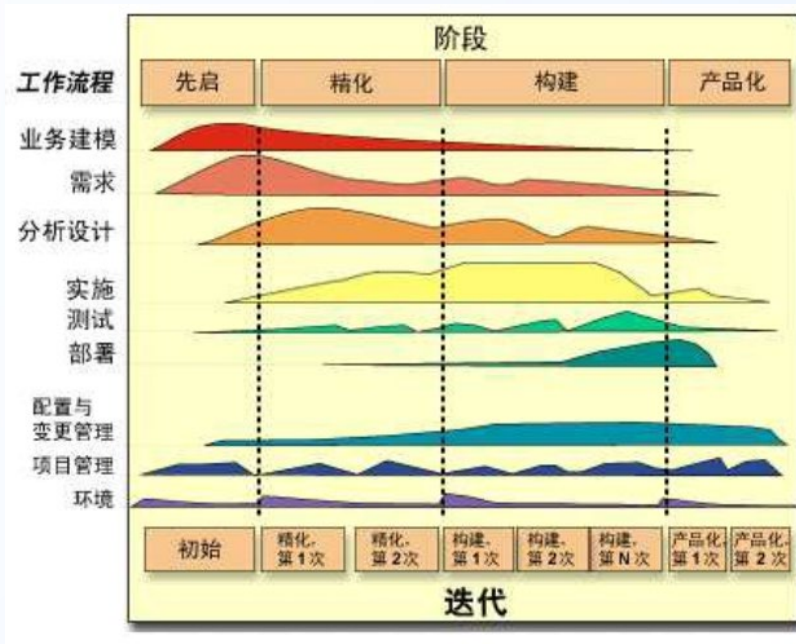
向软件开发机构提供软件开发环境，包括过程管理和工具支持。

3.3 RUP软件开发生命周期

RUP软件开发生命周期是一个二维的生命周期模型，实质上，RUP采用的是“**迭代-增量生命周期模型**”。图中纵轴表示核心工作流，横轴表示时间。

RUP把软件生命周期划分为**初始阶段、精化阶段、构建阶段、移交阶段（产品化）**。

RUP强调采用迭代和增量的方式来开发软件，整个项目开发过程由多个迭代过程组成。在每次迭代中只考虑系统的一部分需求，针对这部分**需求**进行**分析、设计、实现、测试和部署**等工作，每次迭代都是在系统已经完成部分的基础上进行的。每次给系统增加一些新的功能，如此循环地进行下去，直到完成最终项目。



3.4 RUP工作阶段

RUP把软件生命周期划分为四个连续的阶段。每个阶段都有明确的目标，并且定义用来评估是否达到这些目标的里程碑。每个阶段的目标通过一次或多次迭代来完成。

在每个阶段结束之前，都有一个里程碑来评估该阶段的工作成果，如果未能通过评估，则决策者应该做出决定，要么终止项目，要么重做该阶段的工作。

四个阶段是：

- 初始阶段
- 精化阶段
- 构建阶段
- 移交阶段

(1) 初始阶段

□工作目标：判定是否值得开发目标软件产品

□要做的事情：

- 全面理解领域
- 构件业务模型：准确理解客户组织如何在领域内运作
- 确定项目范围：确定目标软件应该包含的内容，它一定是业务模型的一个子集
- 制定最初的业务用例
- 进行可行性分析
- 所提议软件的成本、效率是否合算？
- 所提议软件能及时交付吗？
- 所提议软件开发中存在哪些风险？如何降低这些风险？
- 将参与该软件的团队有必要的经验吗？
- 这个软件产品需要硬件支持吗？需要的硬件能及时到吗？
- 需要软件工具吗？能得到及时这些需要的软件工具吗？

(1) 初始阶段

□得到的成果

- ☆ 领域模型初始版本
- ☆ 业务模型初始版本
- ☆ 需求制品初始版本
- ☆ 分析制品初始版本
- ☆ 体系结构初始版本
- ☆ 风险初始清单
- ☆ 细化阶段计划
- ☆ 业务用例的初始版本（包括：软件的范围描述、财务细节，收益预计、市场评估、初步的成本估算，成本效益分析）

(2) 精化阶段

□工作目标

精化最初的需求、精化体系结构、监控风险并精化风险的属性、精化业务用例，以及制定软件的项目管理计划。

□工作内容

- 完成需求工作流
- 实际执行整个分析工作流
- 完成体系结构设计

□得到的成果

- 完全的领域模型
- 完全的需求制品
- 完全的分析制品
- 更新后的体系结构版本
- 更新后的风险清单
- 软件项目管理计划
- 完全的业务用例

(3) 构建阶段

□工作目标

- 开发软件产品的第一个可操作的版本，即所谓的Beta版。

□工作内容

- 软件产品的实现
- 软件的单元测试
- 软件的集成测试

□工作成果

- 适当的初始用户手册及其他手册
- 所有制品（Beta版）
- 完全的体系结构
- 更新后的风险清单
- 软件项目管理计划（对余下部分）
- 更新后的业务用例（如有必要）

(4) 移交阶段

□工作目标

确保用户需求已得到充分满足。

□工作内容

纠正软件产品中的错误

□完成所有手册

尽力找出先前未识别出来的风险

□工作成果

- 所有制品（最终版本）
- 完全手册



THANK YOU

浙江大学机械工程学院航空制造工程研究所
王 青 李江雄